Chapter 21. How AI Supports SRE

By Boris Arnoux, Chris Wasik, Clément Baïsse, Daniel Marshall, Ioannis Papapanagiotou, Ramón Medrano Llamas, Ruslan Meshenberg, Stevan Malesevic, Thomas C. Hubschman, and William Thurston
with Garrett Holthaus

The integration of AI into Site Reliability Engineering is a fundamental shift in how we engineer, operate, and build production systems. However, discussing how AI is used in SRE presents a unique challenge: everything is changing so fast, it's hard to write down anything we're certain will stay true.

As we write this, many organizations are reporting that increasingly large percentages of their new code is written by AI, and agents are beginning to autonomously navigate complex troubleshooting flows. Just a year ago, AI code generation was helpful but required significant human interaction. Two years ago, agentic tool use by language models was unreliable and brittle. The next year is likely to bring similarly large advances in AI capability and deployment, rendering any specific tool or "how-to" guide written today outdated within months.

Therefore, while this chapter introduces the tools of today, we focus heavily on the durable principles of an AI-first approach. To remain relevant, SRE organizations must prioritize finding opportunities where AI capabilities truly match business problems, and structure teams to adapt as those capabilities evolve.

Complexity, Scale, and Risk

To understand how to apply AI to SRE, we must first appreciate the new pressures bearing down on our systems. Microservice architectures played a key role in scaling our teams and infrastructure, but they came at a cost: interactions between components have become exponentially more complex. Modern services now sprawl across vast geographical topologies, diverse hardware, and an increasingly intricate regulatory landscape. The sheer cognitive load required to understand these systems has become a reliability risk in itself.

This complexity is compounded by an accelerated rate of change. Business demands have long driven a constant stream of updates, but the emergence of AI-driven development has fundamentally altered the equation. Developers can now deliver dramatically more system change, faster than ever before. This velocity is a double-edged sword: as feature delivery accelerates, the rate of failures is likely to as well. We are pushing larger, faster changes into systems that are already difficult to understand.

Standard automation—scripts, runbooks, and simple logic—is excellent at executing known tasks, but it is brittle. It frequently fails when faced with the novel, effectively nondeterministic failure modes of complex distributed systems. AI offers a way through this complexity. It can process unstructured signals, infer intent, and navigate complex topologies without rigid rules.

However, this power comes with a fundamental trade-off. Unlike a script that either succeeds or fails with a specific error code, AI operates probabilistically. It deals in confidence intervals and best guesses, not deterministic execution trees. When we introduce this probabilistic nature into high-scale, high-speed production environments, we introduce the risk of probabilistic failure at scale. At human interaction scales, ambiguous processes are typically noticed and caught after one or a few deterministic errors. With AI, the scale of interaction can be amplified across all of production, and even a 1% error rate can quickly lead to catastrophic impacts. As a result, we cannot simply sprinkle AI on top of existing processes. Given the magnitude of the change, we must rebuild our approach to reliability from the ground up and prioritize a cultural shift within our teams to manage this new class of risk. Many organizations will need to shift from a culture of verifiable correctness to one comfortable operating within a spectrum of quality and safety, embracing nondeterminism.

Structuring for Evolution

To navigate this rapid progression, organizations must cultivate three distinct capabilities.

Experimentation functions as a discovery and validation mechanism. A dedicated portion of the organization should be asking if current AI capabilities match present needs, and then running experiments to confirm they match. This group tests the bleeding edge of model and agent capabilities with the goal of failing fast and identifying the moment a capability becomes viable. At the current rate of change in AI and model capabilities, a failed AI experiment today may be followed by success in only a few months, so it's best to regard experimental results as provisional, rather than final.

Once an experiment completes, the organization must determine if the outcome is ready for *productionization* and broader deployment. When broader deployment is warranted, this involves rigorous engineering to build the guardrails, evaluation pipelines, and integration with existing infrastructure necessary to harden experiments into production-grade reliability tools. Ongoing validation and monitoring of behavior and impact is necessary, as AI capabilities can progress dynamically and unevenly. As an example, a more capable model driving existing processes may take more action than expected in production, leading to undesirable outcomes if not effectively validated and monitored.

Finally, rapid change requires *enablement* and change management. An AI-first transition creates significant human friction, requiring a dedicated focus on training and culture. We must rebuild the intuition of senior engineers accustomed to deterministic systems while upskilling junior engineers who may never triage an incident without an AI assistant.

The Hierarchy of Production Health

We organize our AI strategy around the hierarchy of production health introduced in Chapter 1. This hierarchy prioritizes the foundational elements that remain constant regardless of how quickly AI progresses and how fast change comes. We will explore AI in SRE by following this hierarchy:

1. *Sociotechnical foundations (psychological safety and continuous improvement)*

   Before we can deploy AI agents to manage production, we must establish the governance, safety, and cultural frameworks that allow humans to trust the resulting changes and be in a good position to suggest improvements.

2. *Situational awareness*

   We move beyond simple monitoring to deep understanding, using AI to perceive intent and impact amid the noise.

3. *Continuous preparedness*

   When systems fail, AI acts as a partner for incident management, mitigation, and learning.

4. *Infrastructure design*

   We engineer the environment to be robust, using standardized interfaces, adaptive release engineering, and toil elimination.

5. *Resilient products*

   Reliability is baked into the product from day zero through AI-augmented design and testing.

6. *Emergent reliability*

   The ultimate goal is a system that is reliable by default, where autonomy reduces the need for manual interventions.

Sociotechnical Foundations

A crucial first step in allowing humans to trust AI and improve it is establishing true psychological safety: a blameless environment where engineers are able and willing to take informed risks to advance the state of what's possible in production.

This transformation relies on a positive feedback loop: as we pivot from deterministic automation to probabilistic production management, a strong culture of blamelessness becomes a prerequisite to success. Organizations must avoid the trap of mandating mechanical AI adoption before integrating sufficient safety and governance measures, as doing so undermines both trust and the long-term viability of these practices.

The Role of Humans

As AI becomes the method of choice for reliably and scalably automating workflows traditionally performed by domain experts, concern naturally arises. Engineers worry whether AI will replace them, whether it can match human reliability benchmarks, and whether they can trust nondeterministic architectures during critical outages. These concerns are natural reactions to a radical shift that challenges decades of established reliability best practices. Our current workflows are designed with specific assumptions about human intervention—assumptions that AI often invalidates. To enact such a drastic change, technological evolution is not sufficient; we require sociotechnical change management.

In these early days, given the rapid evolution and improvement of state-of-the-art models, there is considerable uncertainty and discussion about the future role of humans. From human in the loop to human over the loop, will humans get *out* of the loop?

Our hypothesis, which we ground in history, is that AI will not replace humans in the field of SRE: quite the opposite. Humans possess deep domain expertise and, crucially, judgment. AI augments this with machine-speed response times and always-on cover-

age. We are already seeing that domain experts armed with AI insight detect issues faster, diagnose them more accurately, and remediate them more thoroughly than their unaided colleagues. Furthermore, operations that were previously too costly to justify—such as large-scale migrations or broad code refactoring—are becoming viable, offering benefits at scale for a fraction of their pre-AI costs.

The demand for production expertise currently outstrips supply. AI assistance helps bridge this gap by enabling more engineers to assume production responsibilities. It allows junior engineers to integrate best practices into their daily work, acting as a training accelerant rather than a threat. The trade-off here is that AI creates yet another level of abstraction, separating engineers from what's happening on the machine level. AI is fundamentally different from previous shifts in the industry (such as GUIs and IaC). Not only is it a faster-moving transformation, but its impact is not limited to any one practice or industry—we're seeing AI adoption and the resulting changes proliferating rapidly into every knowledge-based domain.

We foresee a dynamic similar to the economic concept of the Jevons paradox, which suggests that as technology increases the efficiency with which a resource is used, the total consumption of that resource rises rather than falls. As AI is showing early promises of reducing the cost of reliability for individual services, organizations are unlikely to reduce headcount. Instead, they will likely consume these efficiency gains by either absorbing the growth of the compute and datacenter resources required to power them, or demanding higher reliability across a broader portfolio of services—either way, increasing the total scope of SRE work.

The role itself, however, will undergo significant changes. Since the creation of the discipline, SREs have been on a relentless quest to automate mundane and repetitive work. AI allows us to make a significant dent in this toil, freeing engineers to focus on the creative aspects of their roles: architecting, designing, and hardening production-grade systems rather than simply operating them. This shift brings SRE closer to its ultimate goal of engineering reliable-by-default systems. Humans will remain essential to mission-critical operations, but instead of executing static playbooks, they will engage primarily at critical junctures where their expertise and judgment are decisive. These human-driven interactions are captured and included in model training and future iterations, ensuring the AI aligns more closely with human judgment. This interaction cycle allows both engineers and AI systems to continuously improve.

The Safety and Reliability of AI Solutions

Introducing AI agents into the production environment fundamentally changes the risk profile of our systems. Unlike traditional software that executes deterministic logic, AI agents operate with a degree of nondeterminism and autonomy. The primary challenge in applying this to production management is the absence of a safe execution environment. In software development, a coding agent can safely make a mistake within a local repository; the worst-case scenario is a failed build. In production, however, an agent interacts with live user traffic, real revenue streams, and critical data. A mistake here—like incorrectly identifying a healthy cluster as unhealthy and draining it—causes an immediate, tangible outage.

This risk is compounded by the sheer volume of interaction. As AI accelerates software development, we anticipate a massive increase in the volume of changes heading toward production—potentially orders of magnitude higher than today. We are already seeing SREs in our teams struggle to keep up with this rate of change with their current processes. In response, our SREs are now using AI in production just to keep up with AI in development. The safety of these production agents is paramount.

To entrust AI with production access, we require a safety architecture built on three pillars: *transparency, contextual awareness*, and *progressive authorization*.

These pillars do not exist in a vacuum; they are extensions of established SRE practices. For example, AI safety boundaries can be directly tied to error budgets. If a service's error budget is depleted, the safety architecture can automatically downgrade the AI from autonomous execution back to a human-in-the-loop advisory role until the budget recovers.

When implementing this architecture, teams should prioritize foundational controls first: strict least-privilege access for AI agents and dry-run auditing capabilities. Only once these day-zero controls are proven should teams layer on complex, dynamic anomaly detection and automated mitigation capabilities.

To entrust an AI agent with production access, first, the AI must be transparent. It is not enough for an agent to execute a mitigation; it must explain its reasoning, the signals it used, and its confidence level. This transparency allows human operators to verify the "why" behind the action in real time. Second, the agent must possess contextual awareness. A specific command might be safe on a Tuesday morning when customer traffic is low but catastrophic during a peak traffic event. Therefore, every action must undergo a

real-time risk assessment that considers the surrounding context—including deployment freezes, error budgets, and active incidents—before execution.

Finally, we rely on progressive authorization. We do not grant agents root access on day one. Instead, trust is earned. Agents typically begin with read-only access, diagnosing issues without the ability to change state. Only after demonstrating consistent success in evaluation pipelines and real usage do they graduate to proposing actions for human approval, and eventually to autonomous execution for low-risk tasks. This graduated approach reduces the risk that the agent's autonomy exceeds its proven competence and ensures that mistakes made and the human corrections applied feed into the process of continuous improvement.

This dynamic evaluation system treats AI performance as a service reliability problem. AI helpfulness and accuracy can be tracked using standard SLOs. For example, an AI agent might have an SLO for "mitigation proposal latency," while safety constraint violations consume an "AI error budget" that, if exceeded, pauses autonomous operations.

Even with mature agents, absolute human control remains essential. We require break-glass mechanisms to immediately block an agent's access to production if it begins misbehaving. We have found engineers are more able to engage with agents confidently and in a psychologically safe way when they have the ability to quickly stop any actions that are underway or planned.

Crucially, we introduce the principle of *reversible operations*: a production action is not taken unless the agent knows how to undo that action. If an agent pushes a configuration change to fix latency, it must immediately monitor the result over an appropriate, context-aware time window. If the service fails to reach expected health thresholds after accounting for normal warmup periods, it must autonomously trigger the inverse operation to restore the previous state. The action, validation, and potential rollback are treated as a single transaction that either succeeds completely or is fully reverted. While the principle of reversible actions is foundational to SRE, in practice—and especially across the broader industry—widespread adoption of the rollback approach remains rare. The industry norm is often to rely on "fixing forward" during crises. Paradoxically, AI agents provide the forcing function to finally implement strict reversibility: because agents lack the human intuition required to safely navigate a fix-forward scenario, the requirement for automated, guaranteed rollbacks becomes absolute.[1]

The reliability of these safety mechanisms depends entirely on the quality of the underlying technical foundations. An agent is only as good as the data it perceives. Therefore, we must ensure high-fidelity access to system topology, dependency maps, service life-

cycle events, and historical incident data. This is often managed via real-time retrieval-augmented generation (RAG) platforms, which provide the agent with the immediate state of the world at the time of action. Additionally, agents must interact with the world through clean, standardized interfaces. Tools should be exposed via protocols like the Model Context Protocol (MCP), which allow agents to interact deterministically with infrastructure APIs. As we move toward multi-agent systems, we also require standardized Agent-to-Agent (A2A) communication protocols, allowing specialized agents to coordinate with generalist agents without human mediation. Orchestrating these collections of agents—reliably launching, monitoring and connecting them—emerges as a first-class infrastructure challenge that must be solved to operate safely at scale.

Ultimately, we must close the loop between operation and improvement. Ensuring reliability requires moving beyond static test suites to dynamic evaluation systems that assess agents on safety, helpfulness, and accuracy. Just as a traditional service uses an availability SLO (e.g., 99.9% uptime) and an error budget to govern deployment velocity, an AI agent should have a helpfulness SLO (e.g., 95% of suggestions accepted by humans) and a safety SLO. These AI evaluation signals must be integrated into established reliability frameworks. For example, when an AI agent exhausts its safety error budget, it is automatically degraded from autonomous execution back to human-in-the-loop mode, treating AI impact assessment exactly like any other service degradation. This feedback loop (shown in Figure 21-1) allows us to retrain models and refine prompts, helping to ensure that our agents become smarter and safer with every incident they assist.

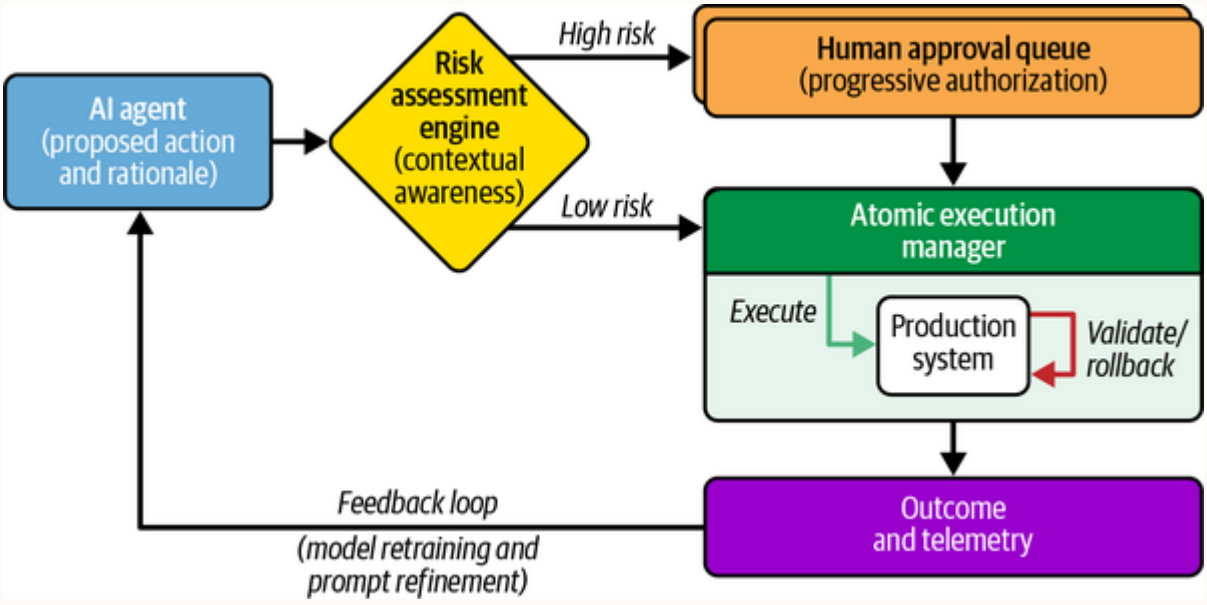


Figure 21-1. AI operations safety architecture and feedback loop

With these safety architectures and feedback loops established, we create the psychological safety and continuous improvement necessary for humans to trust the resulting changes. Only then can we move on to utilizing these agents to achieve superior situational awareness of our complex systems.

Situational Awareness

With the safety foundations in place, the next layer in the hierarchy is knowing what is happening. AI fundamentally changes how we perceive the system, allowing us to move from metrics to meaning. Rather than replacing traditional observability signals, AI synthesizes them, correlating disparate data streams to provide high-level semantic context and actionable insights.

From Metrics to Meaning

Incident detection has traditionally relied on well-established telemetry—metrics, logs, and traces—to identify production issues. While essential, these methods are often blind to issues that are novel, subtle, or emerging, or that primarily manifest as user-perceived problems rather than clear deviations in observability signals. Pre-AI detection systems are typically overbroad or scoped to known failure modes, with alerting logic accumulating reactively over a product's lifetime. This strategy scales poorly; humans cannot predict every new failure mode before it occurs.

Furthermore, what you measure isn't always what the customer experiences. Without granular knowledge of every customer's use case, SLOs remain approximations. A 0.1% error rate may seem negligible in aggregate, but for a customer that calls an API 1,000 times per operation, that rate guarantees failure in almost every attempt. There may also be customers whose requests are serviced in a way that leads to much higher error rates than our high-level dashboards reveal. For example, most of a 0.1% error rate might originate from just 1% of customers with a unique usage pattern that fails 10% of the time. It is in these context-dependent gaps—where metrics look green but users are in pain—that AI has the highest potential to revolutionize our situational awareness.

In "Implicit SLOs", we discussed how customer expectations anchor around historical performance. Consequently, a great deal of research was applied to anomaly detection using traditional machine learning techniques to close this gap. However, as illustrated in Figure 21-2, distinguishing benign anomalies from true failures proved difficult in noisy production environments. A new feature launch, a regional traffic migration, or load-shedding mechanisms kicking in can all look, mathematically, like anomalies. These statistical approaches failed to find widespread adoption because they lacked an understanding of user intent—did the user actually get the outcome they were expecting?

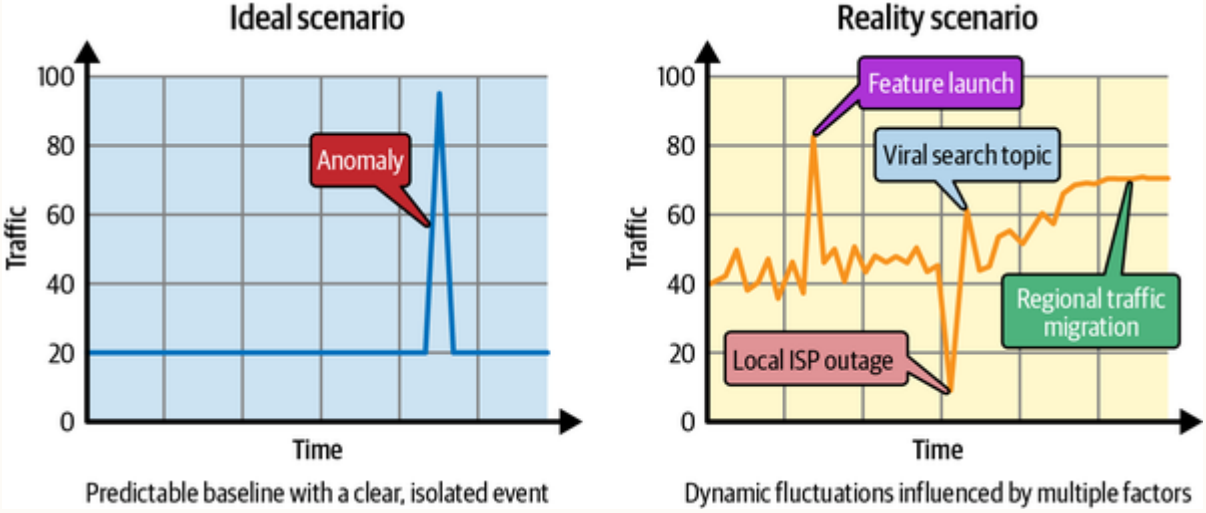


Figure 21-2. Network traffic over time: ideal versus reality

One often untapped data source aligns well with user intent: written customer feedback. Historically, processing unstructured feedback from support tickets, social media, and forums was too slow to be an operational signal, but language models have fundamentally changed this dynamic. Unlike traditional metrics, this information is rich in context, often taking the form of "I tried to do X, with environment Y, and got outcome Z."

By clustering feedback based on common properties like time range, operation, environment, or region, AI provides a highly reliable signal for emerging outages. The semantic reasoning capabilities of language models allow them to piece together fragmented reports that individually lack sufficient detail but collectively describe a clear failure pattern. These clusters offer distinct advantages over traditional alerts: the volume of the cluster serves as a natural severity signal, and the intrinsic effort required to write feedback acts as a noise filter.

Case Study: Detectr

To illustrate how these concepts move from theory to production, let's look at Detectr, Google SRE's Gemini-powered platform that analyzes and organizes user feedback to detect user-reported outages. This is essential to monitoring modern systems, as many have intermittent failure modes with differential impact affecting only some users and some use cases. As we add AI to these customer-facing systems, they will fail probabilistically and partially. Ultimately, end users are likely to be the first people to detect these targeted, partial failures. Detectr doesn't just provide a way of addressing user feedback; it harnesses that feedback to allow end-to-end quality measurement with grounding in real human users.

Detectr aggregates signals across social media, customer support, product forums, and other human sources. It functions as an important backstop to traditional monitoring and is designed to catch novel issues that slip through the metric-based net. As shown in

Figure 21-3, this is done through a multipass AI pipeline that generates structured outage reports:

1. *Augment*

   Internal information about the system state is attached from the time of the complaint.

2. *LLM filter*

   Irrelevant posts are removed and data is categorized.

3. *Embedding cluster*

   Loose embedding clustering is performed to reduce the search space.

4. *LLM cluster*

   Related reports are grouped together to identify potential outages.

5. *LLM report*

   A structured report is generated for triage.

6. *Notification*

   The report can be delivered through normal mechanisms like email, tickets, or paging.

This process transforms messy, unstructured user feedback into structured, actionable outage notifications that can be sent to the appropriate teams.

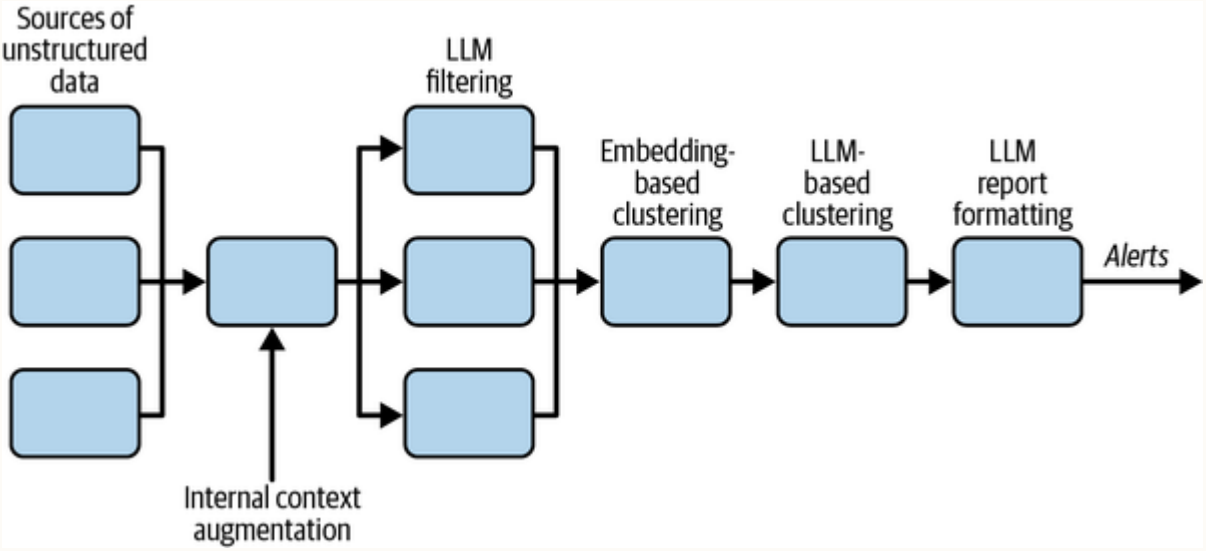


Figure 21-3. A pipeline for scalably processing unstructured data

In early 2026, Detectr matured from a prototype into core infrastructure. It has been adopted by teams across all major Google products and has successfully served as the primary method of escalation for many outages. We estimate Detectr's value with measurements taken during Q4 2025 and Q1 2026—it reduced the impact of these events on customers by thousands of cumulative hours, thanks to earlier detection and deeper understanding. As Ruslan Meshenberg, VP of SRE, commented:

> *I think Detectr has been one of the most meaningful projects/products under the AI Ops Program this year. [It's] already shown significant business impact for multiple PAs—YouTube, Maps, Ads, GCP [Google Cloud]—providing significantly earlier detection for issues/outages and thus allowing SRE/SWE teams to address the issues earlier. The combination of revenue saved and reputation of reliability upheld by virtue of using Detectr cannot be overstated. It's on a clear trajectory to become used almost universally across all Google PAs. As the executive sponsor of the AI Ops Program, I think it's becoming one of the biggest breakout hits of 2025.*

This success illustrates the productionization phase of AI adoption we discussed in "Structuring for Evolution". Detectr began as an experiment in semantic reasoning, but its value came from rigorous engineering—building the guardrails, filters, and integration pipelines that turned an experiment into a durable reliability tool.

AI Impact Assessment

Finally, situational awareness requires understanding the *scope* of the impact. Complexity in modern services often obscures the blast radius of an incident, making targeted customer communication difficult.

We use AI to bridge the gap between technical symptoms and customer impact. An AI agent can review an incident summary, map it to the most likely location and service, and correlate it with an asset management system to determine the most likely set of impacted customers. Furthermore, AI-based matching systems can correlate incoming customer cases with active incidents in real time, automatically detecting if a new report is a symptom of a known outage. This moves impact assessment from a post-incident estimate to a real-time operational signal, allowing SREs to communicate clearly with everyone who is affected.

Continuous Preparedness

Once we know the system is broken, we must fix it. This layer covers the reaction to failure: managing the complexity of the incident, mitigating the impact, and ensuring we learn from the failure to prevent recurrence.

Managing Incident Complexity

A well-executed incident response is a race against business loss and customer impact. In our most impactful incidents, it is common for the number of responders to grow, and with it the communication overhead. We want everyone involved in an incident to understand their role and concentrate on their assigned task. Reducing overhead and distraction is an important goal, and AI can help significantly here.

Instead of asking the incident commander to manually summarize the state of the world for new joiners, AI agents monitor the communication surfaces used during the incident, such as traditional incident response tools, chat spaces, videoconferences, and tracking documents. They then consolidate and summarize this stream of information to ensure the right information is shared at the right time. Furthermore, agents support increased context sharing between humans participating in the incident (e.g., during shift changes), preventing context loss and eliminating the toil of manually creating handoff documents.

The need for dynamic, real-time, actionable context extends beyond communication to the tools we use during incident response. Historical monitoring dashboards were often static, displaying a predefined set of metrics based on rules defined months or years previously. This approach does not tailor to specific incidents and rarely provides a single view that responders can use to understand the current state. As a result, the people working on the incident typically needed to open multiple dashboards—each with multiple graphs that might or might not be important—and manually interact with them based on dimensions relevant to the incident, such as location, time, or customer segment.

To manage this complexity and speed up understanding during an incident, we use AI to generate incident-specific insights and tailor the set of graphs on the fly. The AI agents use multiple approaches to decide what graphs need to be on the dashboard:

Historical similarity

Using embeddings on the incident context and comparing distance to historical incidents to find graphs that were useful in the past

Playbook context

Extracting graphs mentioned in the playbooks triggered by the alert

Intent matching

Selecting graphs that meet the logical intent of the incident description

Once the agent has created the set of dashboards, the responder is presented with suggestions of relevant dashboards and views that show impact, related systems, and potential early signals of trouble. Because agents can search quickly across many signals and many sources of context, this capability has driven a measurable reduction in the time it takes to mitigate incidents across Google.

Mitigation Versus Root Cause Analysis

When applying AI to incident response, it is important to distinguish between two distinct phases with different goals: *mitigation* of impact and *root cause analysis of contributing factors*, where we search for deeper understanding.

For many services, finding the right mitigation strategy is often a process of selection rather than discovery. At Google, our incident response AI system analyzes alert types, error codes, system metrics, time of day, and system characteristics, and identifies the most appropriate action from a predefined set, such as "rollback," "drain," or "upscale." In this phase, deep understanding is secondary to speed. Determining which mitigation is effective and safe is the primary concern. By presenting a high-confidence recommendation to the human operator, like "95% confidence that a rollback will mitigate,"[2] the AI provides a practical path to reducing incident and customer impact while ensuring our human experts apply judgment.

In contrast, root cause analysis requires deep investigation. Here, the AI acts as a detective, generating hypotheses based on system topology and dependencies to identify where an incident may have originated. It traces anomalies back through distributed systems to identify the source of errors, automating the tedious work of sifting through logs. To do this effectively, the AI uses observability data to ground the investigation in fact, topology data to establish the domain of the problem, and anomaly detection systems to pinpoint deviations.

Learning from Failure

The final, and arguably most important, step of preparedness is ensuring we do not repeat the same mistakes. We leverage rich historical data from incidents to drive continuous improvement and systematically eliminate risk through structured learning.

This process begins with the incident report. As explained in Chapter 15 of the first edition of this book, incident reports support a culture of learning from failures, but creating them involves the manual toil of collecting timestamps, chat logs, and graphs. We leverage AI to remove this tax on learning by automatically drafting incident reports, tagging risks, and suggesting action items. This ensures that the documents are created faster and follow a consistent structure, enabling humans to spend their time discussing the lessons learned rather than formatting the timeline.

We can extend this learning beyond individual documents to the entire corpus of historical data. AI processes historical incidents to derive metainformation such as common root causes, successful mitigations, and violations of production principles. This aggregate view ensures that appropriate risk tagging—which humans can easily miss—always exists, driving smarter engineering investments.

This intelligence ultimately feeds back into the code and documentation. AI can identify clusters of similar issues to drive effective deduplication, preventing multiple engineers from investigating the same bug, and can even suggest code-level fixes based on similar past resolutions. Taking this insight gathered from data, we optimize our playbooks to ensure our operational knowledge endures. Playbooks often become stale, drifting out of sync with code and topology. To address this drift, AI systems continuously review existing playbooks against real-world usage during incidents. If an incident occurs for which no playbook exists, the system uses the incident data and the human actions taken to autogenerate a draft playbook, ensuring that the next time the pager fires, the on-call engineer is better prepared than the last. Currently, many individual systems are delivering on small portions of the overall pictures, but the end goal is to create an autonomous end-to-end experience.

Infrastructure Design

This layer is about engineering the environment the service operates in. It is a powerful tool to improve production health, covering release procedures, testing, and the elimination of toil. The approaches shared can be applied independently of each other, and

the order in which you pursue them should be driven by your organization's specific needs and priorities.

Adaptive Change Management

SRE is commonly involved in decisions around deployment: choosing locations, provisioning dependencies, and setting initial scaling factors. Historically, these decisions relied on human intuition and static heuristics. Today, we use AI to connect signals from service taxonomy, dependency health, and historical demand to extract optimal deployment values mathematically.

This shift extends to the safety of deployments. Minimizing customer impact during a change is a critical SRE responsibility, often managed by static policies like "wait 30 minutes between deployments to different failure domains." However, static policies are often too slow for urgent fixes, while being too fast to stop subtle regressions. We are seeing success in using AI to correlate signals and adaptively change the speed of a deployment through its own risk judgments, dynamically optimizing the balance between safety and velocity.

Once deployed, the environment must be maintained. Modern platforms provide a large amount of diversity in possible configuration options, and demand on our services is often dynamic, making a service's optimal configuration a moving target. We use AI systems to continuously review running infrastructure against best practices. By interpreting high-level availability policies, these systems can generate and apply code-level configuration changes at scale, ensuring that the fleet does not drift from its intended design.

Eliminating Toil

The first edition of this book defined toil as work that is manual, repetitive, automatable, tactical, devoid of enduring value, and scales linearly with service growth. While most of this definition continues to hold true, AI now offers us the ability to automate the automation process itself.

The first challenge in reducing toil is seeing it clearly. Human identification of toil is prone to recency bias—we remember the annoying task from this morning, not the chronic time sink from last month. We now use AI to perform real-time analysis of ticket streams, reviewing human actions, investigation steps, and mitigation strategies. By clustering this data, the system identifies high-volume areas ripe for automation that humans might miss. Even if you do not have a well-defined system to track tickets and

unplanned work, AI is very capable of deriving insights about unplanned work from emails, chats, docs, etc.

Simultaneously, we tackle the noise at the source. An AI ticket interceptor reviews incoming issues, finding similar past tickets and calculating the probability of duplication. Aggregating these duplicates improves the quality of handling, giving the human owner better context and drastically reducing the volume of busywork in the queue.

Once a cluster of toil is identified, we move to automation. AI generates playbooks from the successful actions taken by humans in the past. An agent then processes these playbooks, executing the steps needed to address new tickets autonomously. While autonomous execution is powerful, guardrails need to be put in place to determine what actions are allowed to be fully autonomous and when a human in the loop is needed in terms of reviews and approvals. If the agent encounters a novel situation, it falls back to requesting human help.

Crucially, this system includes a self-tuning loop. If a ticket resolved by an agent is reopened by a human, it enters an evaluation phase. The system analyzes why the automated fix failed and retunes the playbook, ensuring the agent has better context available the next time it takes action.

For toil that cannot be fully automated, AI focuses on enrichment. Historical data shows that a significant portion of ticket resolution time is wasted searching for missing information. We have built systems that process historical interactions to understand what context is required for different issue types. When a new ticket arrives, the system intercepts it, checks for necessary information, and immediately asks the submitter to fill in the gaps before transferring ownership to an SRE. This simple step eliminates the time-consuming back and forth of context gathering.

Verifying Resilience

Finally, infrastructure design requires proving that the system can survive failure. Natural disasters, cyberattacks, power events, and cooling failures are inevitable; survival is engineered against those constraints. AI is transforming how we decide what to test, how we review plans, and how we analyze results.

We classify resilience tests according to a structured taxonomy of resilience properties, such as datacenter isolation or traffic management. This taxonomy allows us to partition the comprehensive scope of factors required to validate service resilience.

Often, test authors start with an organic idea for a specific failure scenario rather than a formal property, so the specific resilience property they are evaluating may not be immediately apparent. To address this, the AI reviews every draft test plan and suggests the properties best aligned with the specific scenario. By automatically categorizing tests, it saves time for authors and ensures that data accuracy remains high across the entire corpus of resilience tests.

Google employs a rigorous review and approval process so that tests run in production are sufficiently safe. This often involves a series of preliminary tests that seek to incrementally increase the blast radius of a failure. To help onboard new test authors to this process, the AI conducts a preliminary review of all test plans against established safety guidelines and identifies potential deficiencies that might expose systems to unnecessary risk (such as an overly aggressive deployment strategy, insufficient user segmentation, or the lack of lower-risk preliminary tests). This automated preliminary review reduces the burden on expert human reviewers by ensuring that only sufficiently well-formulated test plans are presented for their assessment.

After tests are run, the AI analyzes the results by performing a comparative analysis between the steady-state behavior of the service and the behavior observed during the test. Depending on the specific resilience property under evaluation, the model selects distinct metrics to validate that the test produced a measurable impact while confirming that the system survived.

For example, if a test is designed to verify that a system allows for the loss of a whole datacenter, the results must show that traffic served by the downed cluster dropped to zero and that other clusters picked up the resulting load. In this case, while aggregate global traffic metrics might remain constant, the AI verifies the traffic shift across clusters during the execution window. This confirms that the system was resilient to the specific fault introduced.

Perhaps the most powerful application of AI in this space is determining where the largest resilience gaps exist. We feed the system recent incident data to identify which resilience properties could have mitigated those specific outages. We then compare this data against the set of recently executed tests to surface gaps in our testing. This identifies precisely where resilience must be engineered and where the incident needs to be replayed with new protections in place.

In addition to the insights into testing gaps, proposals often come from centrally defined resilience campaigns that seek to scale lessons learned from a single system to the

broader organization. Regardless of the source, once models have identified areas for validation, we present these suggestions to teams to target their investments.

Prioritizing the verification that past failure scenarios have been remediated alongside centrally directed resilience campaigns is still a largely human-driven exercise. This involves weighing the cost and desired depth of verification, the likelihood of incident recurrence, and impact of the failure. Complex or high-impact failures may warrant larger-scale verification efforts, while smaller incidents may only necessitate more localized testing or verification. We are also experimenting with AI-driven meta-analyses of incidents across teams or larger groups, helping teams focus their testing budgets on the highest-impact verification efforts.

Finally, we move from identification to generation. Rather than simply asking teams to address a gap, we are experimenting with having AI draft detailed test plans for them to review and execute. As the corpus of available tests and templates grows, models use them as inspiration, leading to higher-quality and more impactful test plans being validated across the fleet.

Resilient Products

Production health starts at the very beginning of software development. Delivery of resilient products is the fundamental goal; the earlier reliability practices are applied, the more effective our efforts become. In practice, this resilience is defined and measured using SLOs that closely track user experience metrics, such as request success rates, latency percentiles, and data freshness. By anchoring design decisions to these concrete SLOs, teams can quantify the impact of architectural choices and ensure that reliability is an engineered property rather than an afterthought.

AI as an SRE Force Multiplier

AI represents the next step in SRE automation, evolving from basic workflow automation to autonomous systems requiring minimal human intervention. These systems establish a continuous learning loop: post-incident analyses, real-time production metrics, and historical performance data are continuously fed back into the AI models. This ensures that the AI-driven design and review processes are grounded in real-world production realities, helping to prevent the recurrence of known failure modes. However, there is a misconception that the primary goal of AI is simply to eliminate toil. While reducing operational overhead is an important first step, it has a mathematical ceiling, with the earliest wins being the most valuable.

Automating toil is often the first application of AI in operations, but measuring its success merely in hours saved fundamentally misunderstands the value of the AI tools. If an SRE team maintains a healthy toil baseline of 25%, even a massive 5× increase in processing that toil will yield only a modest improvement in traditional engineering productivity. Beyond that improvement, AI makes some existing best practices ROI-positive to deploy, leading to increased standardization, security, and reliability. However, the true promise of AI in operations is not about doing the same work with fewer people; rather, it's about opening up entirely new capabilities. With the ever-growing scale and complexity of SRE-managed systems, AI acts as a cognitive lever allowing SREs to perform the kind of systemic, proactive engineering work that was previously out of reach.

The logical next step is to tap into the immense opportunity that lies in scaling our ability to design autonomous and reliable systems in the first place. Beyond daily maintenance, the majority of Google SRE work is in developing and transforming large systems. By applying AI to these human-driven tasks—designing new components, managing migrations, and reviewing code—we unlock a force multiplier that far exceeds the gains from simple ticket automation.

AI Design and Launch Reviews

SRE teams play a key role in reviewing design documents to ensure they meet production standards. This process often requires significant human effort, and the quality of the outcome tends to depend heavily on the experience level of the specific reviewer.

To address this, we created an AI design review system. This system is based on a model trained on our internal knowledge base of reliable system designs, as well as specific insights from past incident reports. It scans design documents to check that known production principles are addressed and known risks are avoided. While this approach does not eliminate the need for human judgment, it significantly improves throughput by performing a first-level review. It enables humans to spend less time on basic compliance checks and more time on complex architectural decisions, while ensuring that even low-risk designs meet a minimum bar of quality without requiring a human bottleneck.

Similarly, we augment the launch review process. Compliance for launches is often a tedious task involving checklists and manual verification. We now use AI agents to filter and raise signals from the actual production prototype being considered for launch. By automatically analyzing the prototype's behavior against reliability standards, the agent makes reviews more thorough and cost-effective, preventing fragile services from ever entering the fleet.

Finally, as models improve, we are moving from reactive review to proactive exploration. We are experimenting with AI systems that autonomously scan large, complex codebases to detect risky patterns and critical weaknesses before they manifest as incidents. This scales the impact of our incident reports: instead of a lesson learned being applied to one service, AI can identify every other service in the fleet that shares the same vulnerability, and proactively open a bug to fix it. This is the beginning of true self-healing design, where the system improves itself based on the collective experience of the entire organization.

Emergent Reliability

The ultimate goal is to make production health inherent to all infrastructure, development, processes, and culture. Software becomes reliable by default.

The Path to Autonomy

Adopting AI in SRE is not a binary switch; it is a structured journey from standard human-operated tooling to fully autonomous systems. To guide this evolution safely and effectively, we use a maturity model, as illustrated in Table 21-1. This model helps achieve *emergent reliability*, where systems become more reliable by design, minimizing manual interventions. The levels are defined by the degree of automation across key operational functions: monitoring, investigation, approval, actuation, and self-directed operations.

Table 21-1. Levels of autonomy

Action → Level ↓	Monitor	Investigate	Approve	Actuate	Self-direct
L0: Manual	Automation	Human	Human	Human	Human
L1: Assisted	Automation	Automation	Human	Human	Human
L2: Partial	Automation	Automation	Human	Automation	Human
L3: High	Automation	Automation	Automation	Automation	Human
L4: Full	Automation	Automation	Automation	Automation	Automation

The five levels of increasing autonomy are:

L0: Manual Execution

All activities are human-driven. Humans must *investigate* alerts, *approve* any course of action, *actuate* the changes, and handle any *operational resolution* that is

required.

L1: Assisted

Automation extends to *monitoring* and *investigation*. The AI agent can analyze data and provide insights or suggestions (an AI-driven incident hypothesis), but a human is still required to *approve* any action and manually *actuate* it.

L2: Partial Autonomy (Human Approval)

Systems can *monitor*, *investigate*, and even *actuate* changes. However, a human must explicitly *approve* any plan before the system proceeds with an actuation. Partial Autonomy is called for when a safety check fails, or it can be a short-lived safety net as we build trust in the AI system.

L3: High Autonomy

Automation covers *monitoring*, *investigation*, *approval*, and *actuation* for specific, well-defined scenarios. The AI agent can independently detect, decide, and act within predefined bounds without human approval, though humans are often notified. Humans still handle novel situations requiring *operational resolution*. Safety relies heavily on technical controls and guardrails.

L4: Full Autonomy

An AI agent can:

- Devise and execute a sequence of actions to diagnose, mitigate, and resolve incidents—especially those not covered by simple, linear playbooks.
- Continuously monitor the impact of its interventions on the system's state and performance.
- Adapt its strategy in real time based on the observed outcomes. This includes the ability to attempt alternative mitigations if initial actions are ineffective, or to initiate rollbacks if an action has adverse effects.
- Manage the entire incident lifecycle until the system is confirmed to be back in a stable and desired state, going beyond just the initial trigger and actuation.

As systems move toward higher levels of autonomy, the potential risks and failure modes associated with autonomous actions, particularly in complex production environments, increase significantly. These risks must be managed through rigorous testing and strict technical guardrails.

Moving a system to a higher level of autonomy is a significant decision that must be gated by rigorous evaluation, demonstrated reliability, and robust safety mechanisms. Prerequisites for progressing to higher levels include comprehensive test coverage, well-defined rollback mechanisms, established error budgets, and a history of successful operations at the current level of autonomy. Key safety principles include risk-scaled rigor, thorough evaluations, secure APIs, auditability, and clear human oversight and override capabilities.

It is critical to remember that the goal is not for every system to reach L4. The appropriate level of autonomy depends on the specific context, balancing benefits like increased reliability and reduced toil against potential risks. When determining the appropriate level, consider factors such as system criticality, organizational risk tolerance, operational maturity, and the cost of failure. An AI system at a lower level of autonomy, when properly implemented, can provide substantial value and may be preferable to a higher-level system if the risks outweigh the benefits.

Case Study: AI Operator

To better understand the journey through SRE autonomy levels, let's look at AI Operator, an initiative within Google SRE to automate the handling of routine and predictable production incidents. AI-driven operations tools like AI Operator are becoming essential to managing the scale and complexity of modern systems.

AI Operator functions as an AI agent designed to be the first responder to production alerts. It ingests alert signals and leverages a set of extensible modules to perform multiple investigations. Its reasoning process (as shown in Figure 21-4) is guided by examples derived from how human experts have effectively investigated similar past incidents. This allows the agent to form and test hypotheses to perform root cause analysis. Once the analysis is complete, AI Operator can select from a catalog of available mitigation actions. Based on preconfigured permissions, it then executes the chosen mitigation.

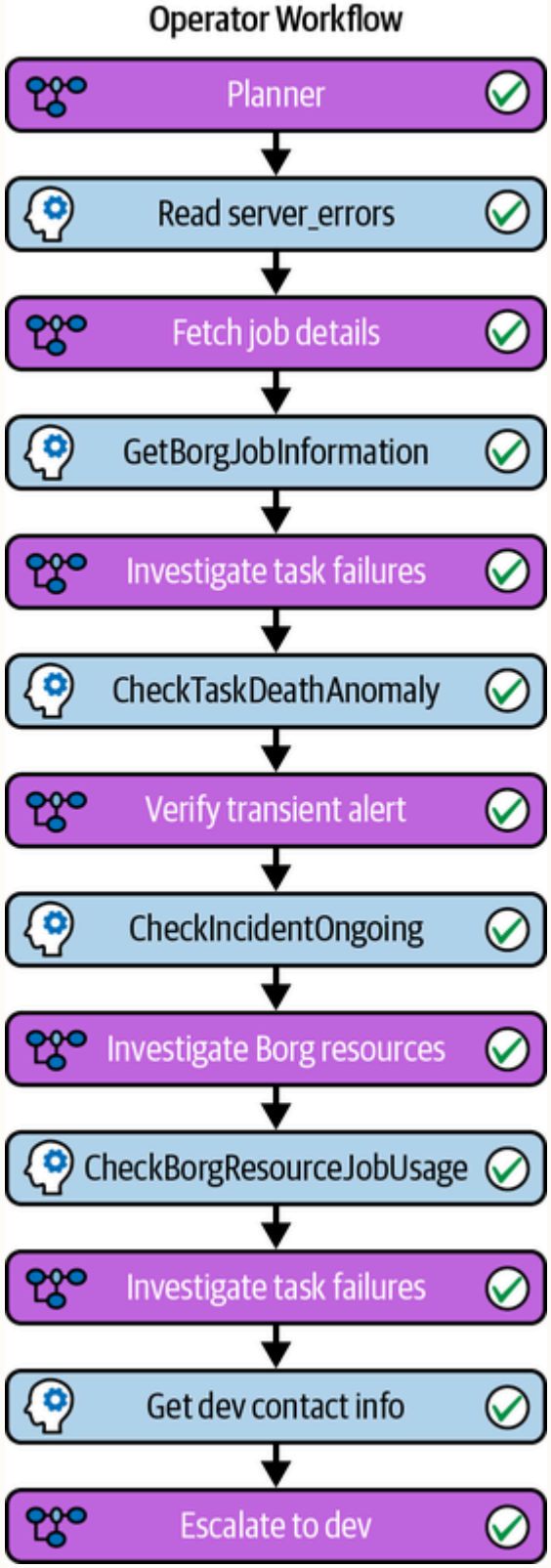


Figure 21-4. AI Operator chain of thought

AI Operator's path through the SRE autonomy levels illustrates a safe and progressive approach to AI adoption in production:

Level 1: Assisted

From the outset, AI Operator provides assistive capabilities. It automatically performs *monitoring* and *investigation* on incoming alerts, correlates signals, checks for similar past incidents, and summarizes its findings. This gives human on-callers immediate context and potential diagnostic paths, accelerating their understanding. The on-caller still handles *approval*, *actuation*, and any *complex planning*.

Level 2: Partial Automation

After its investigation, AI Operator proposes concrete mitigation actions with clear justification. These actions can range from draining network traffic from a cluster to rolling back a software or configuration update. The system handles *monitoring* and *investigation* and can *actuate* the changes, but requires human *approval*. The exact commands and the reasoning are presented to a human SRE, who must review and explicitly approve the action before it is executed. *Complex planning* remains manual. This human-in-the-loop model provides safety while still significantly speeding up mean time to mitigation for common incident patterns.

Level 3: High Automation

AI Operator provides Level 3 capabilities for specific, well-understood scenarios. These are low-risk actions where the system handles *monitoring*, *investigation*, *approval*, and *actuation*. Examples include autonomously restarting a single aberrant task within a large job or clearing a specific type of cache known to cause transient issues. Promotion of any action to Level 3 requires rigorous validation in a dedicated evaluation framework, demonstrating a very high success rate and excellent safety record during Level 2 operations. Humans are typically informed of actions taken. *Complex planning* for novel issues remains a human task.

Level 4: Full Autonomous

The long-term vision for AI Operator is to achieve Level 4 autonomy across a wide range of scenarios. This involves enabling the system to autonomously handle complex tasks within its operational domain, such as sophisticated traffic management and multistep mitigations, from end to end. Reaching this state of full automation for advanced use cases is a significant goal, contingent on further advancements in AI reasoning, real-time risk assessment, and provable safety guarantees.

AI Operator has self-improving capabilities. Through an evaluation framework that analyzes incident metadata and compares automated actions against ideal human responses (referred to as *golden data*), the system can identify areas for improvement. Techniques like LLM-as-a-judge help refine its investigation logic and decision making over time, creating a virtuous feedback loop.

Figure 21-5 shows the evaluation framework in action. On the left side, the agent correctly diagnosed the problem and mitigated it. On the right side, the agent incorrectly diagnosed the problem and therefore failed to mitigate it. Our LLM judge generated a critique and provided a suggestion of how to improve the agent.

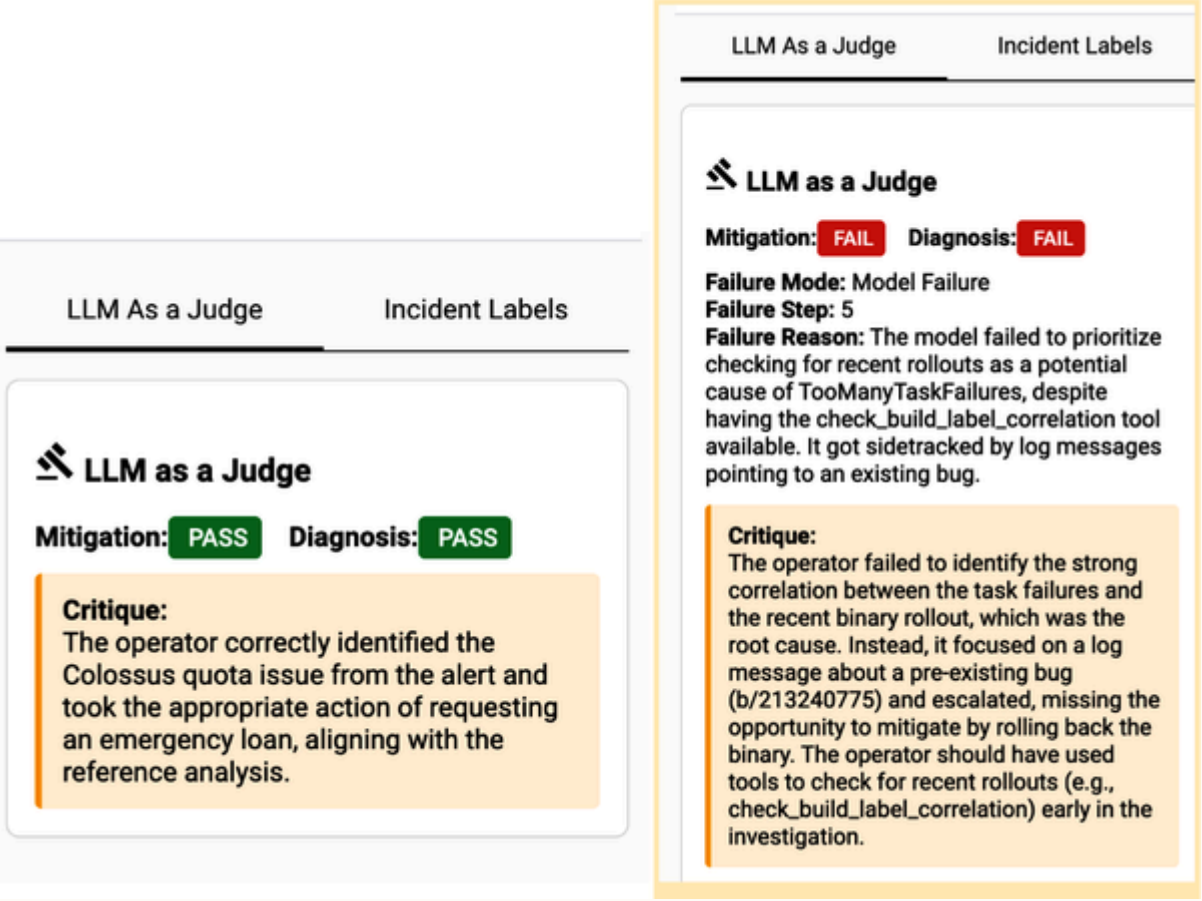


Figure 21-5. Using AI Operator's LLM-as-a-Judge framework

The impact of AI Operator is already being felt, and it is shifting the very nature of on-call work. Rather than simply acting as a faster set of hands for routine tasks, it serves as an analytical partner capable of instantly synthesizing signals across vast, interconnected infrastructure layers. Delegating the overwhelming volume of data correlation and initial hypothesis testing to AI empowers human engineers to operate at a higher level of abstraction. Teams adopting the tool are resolving incidents more quickly; perhaps more importantly, they are gaining the capacity to tackle novel reliability challenges, design resilient architectures, and safely manage a scale of operations that would otherwise outstrip human capability.

The Organizational Shift

Achieving emergent reliability requires more than just new technology; it requires a fundamental shift in business processes, incentives, and culture. Every journey will be different, as these challenges are deeply rooted in the specifics of an organization. There is no specific framework or recipe that will work universally. To illustrate the magnitude of this challenge, we can look at how Google SRE has been evolving since the first edition of this book was published, and how AI is fundamentally reshaping the organization today while keeping it anchored to its original mission.

Since its inception in 2003, the SRE organization has continually reinvented itself to adjust to the growing scale and complexity of our systems. This evolution was driven by product needs: the early days of Search and Ads forged the premises of error budgets, Gmail introduced the requirement for stateful user data management, the explosion of YouTube pushed the boundaries of networking, and the launch of Google Cloud shifted us from managing internal infrastructure to user-facing B2B solutions.

Technological step changes also drove adaptation. The shift to mobile, for example, forced SRE to extend its focus beyond server-centric SLOs to include client-side reliability and to adjust change management strategies for app store release patterns. However, these evolutions were largely organic. The integration of AI represents a unique disruption because it challenges the operating model we have spent the past 10 years perfecting.

For the last decade, Google SRE focused heavily on *platform engineering*. We acknowledged the inefficiencies of fragmentation and prioritized standardization, centralization, and aggressive deduplication. We built a culture where standardization was the gold standard.

AI disrupts this cycle. The transformation is happening toward the end of a normalization cycle, forcing us to pivot when our organizational inertia is pulling toward stability. The pace of technological change is unprecedented: while personal computing performance roughly doubled every two years, the capabilities of applied AI have improved by at least an order of magnitude in the last two years alone. Integrating this superexponential acceleration into a five-year strategy requires us to think very differently about how we organize.

To find the right product-market fit for AI, we had to encourage individual teams to experiment closer to their specific products. We intentionally sponsored incubator structures and accepted the duplication of solutions—anathema to traditional SRE thinking—to maximize learning velocity. We defined graduation paths to promote local solutions to broader use only after they had proven their value. This decentralized approach allows us to navigate the standardization versus innovation polarity by letting the most effective patterns emerge from the bottom up.

This shift creates a significant human challenge. We found that the primary hurdle was not teaching junior engineers to use AI; it was rebuilding the intuition of our senior engineers. In a world where a human mistake can bring a service down, SREs are rightfully conservative. The idea of having fundamentally probabilistic systems interacting with complex production environments is scary.

To overcome this, we had to help some of our most senior engineers learn new habits and techniques. We needed to build trust and confidence iteratively, training people to use the tools of today to build the tools of tomorrow. This is not just about learning to use a model; it is about building confidence in changing daily behaviors, and using a rigorous approach to understand and mitigate any novel risks that are emerging from the new patterns.

Enabling this transformation involves standard engineering, data pipelines, and integration rather than pure modeling. At Google's scale, the challenge is meeting users where they are. Rather than building a single, monolithic AI approach, we treat the transformation as a collection of microfeatures integrated into existing workflows.

One of the main challenges of SRE remains identifying and solving problems that occupy the ideal intersection of being worth the effort and being technically achievable. Just like 10 years ago, the path to success lies in identifying achievable targets and iterating. This pragmatic approach of chipping away at complex problems is better achieved with strong domain expertise. SRE was built on the premise that you hire engineers to do the work, and then to engineer it away. This principle remains true today. AI simply expands the definition of *what* can be engineered away.

Conclusion

The integration of AI into Site Reliability Engineering marks a fundamental shift, transforming both the practice and the systems we manage. It is a transformation occurring at a velocity that defies our traditional planning cycles, forcing us to rethink how we build and operate software.

This chapter describes how we have navigated this shift by organizing our strategy around the hierarchy of production health:

Sociotechnical foundations (psychological safety and continuous improvement)

: Establishing the governance, technical guardrails, and cultural frameworks necessary to trust AI in the loop

Situational awareness

: Moving beyond static metrics to true semantic understanding, using tools like Detectr to perceive user intent amid the noise

Continuous preparedness

Positioning AI as a partner in incident management, using real-time dashboards to mitigate impact, and transforming root cause analysis from a search problem into a solved problem

Infrastructure design

Engineering the environment to be robust by eliminating toil through autonomous agents and verifying resilience through AI-generated testing

Resilient products

Baking reliability into the product from day zero by using AI as a force multiplier in design and code review

Emergent reliability

The ultimate goal, where autonomy and reliability become inherent properties of the system, reducing the need for manual intervention

The landscape of AI changes rapidly. By focusing on the enduring principles of human augmentation, measured autonomy, continuous evaluation, and the foundational reliability of the AI systems themselves, this chapter provides a stable framework for SRE teams to navigate this new era and continue to deliver highly reliable services.

1 Note that this implies additional work is required in the case of destructive migrations.

2 If the confidence is below a certain threshold, it's best to not present the mitigation at all to the on-caller.

Previous chapter
< 20. How SRE Supports AI

Next chapter
22. SRE in Diverse Environments >